

Universidade Católica de Brasília
Análise e Projeto de Software

AnalizadorGit

Documento de Interface e Prototipagem

Versão 2.0

Grupo: Maria Clara, Maria Fernanda, João Gabriel e Luiz
Guilherme

Histórico de revisão

Data	Versão	Descrição	Autor
01/05/2026	1.0	Elaboração do protótipo de alta fidelidade e definição da interface.	Maria Clara G. M. Viana
01/05/2026	1.0	Elaboração do fluxo do usuário e do diagrama	João Gabriel Leite Neirelli
07/05/2026	2.0	Atualização e aprimoramento do documento de interface do Sistema.	Maria Fernanda L. R. De Sousa

Índice

Objetivo da Prototipagem	4
Escopo do protótipo.....	4
Funcionalidades Inclusas.....	4
Funcionalidades Excluídas	4
Limitações Conhecidas	4
Protótipo das Telas.....	6
Tipo de Protótipo	10
Ferramentas Utilizadas.....	10
Descrição das funcionalidades	10
Fluxo do Usuário.....	10
Fluxo de instalação (Setup)	11
Fluxo Principal de Execução (Commit).....	11
Requisitos do sistema.....	12
Requisitos funcionais	12
Requisitos não funcionais.....	12
Casos de Uso.....	12
Caso de Uso – Realizar Commit Seguro	12
Guia de estilo	12
Melhorias Futuras	13
Conclusão.....	13
Anexos	14

Objetivo da Prototipagem

A prototipagem tem como objetivo simular o fluxo do commit, ilustrando como o analisador funciona seja para validar arquivos e mensagens do usuário até instruir boas práticas ao usuário. Além disso permite representar visualmente o funcionamento do sistema, permitindo validar fluxos, interações e usabilidade antes da implementação.

Escopo do protótipo

O protótipo representa o funcionamento de um sistema de análise de segurança integrado ao fluxo de *commits* do *Git*, com foco na identificação de dados sensíveis e na interação com o usuário por meio de interface em terminal (CLI).

Funcionalidades Inclusas

- Simulação da instalação do sistema (AnalisadorGit);
- Execução automática da análise durante o *commit*;
- Identificação dos arquivos adicionados ao *commit*;
- Detecção de padrões de dados sensíveis;
- Classificação de risco (alto ou médio);
- Exibição de mensagens no terminal;
- Indicação de arquivo e linha do problema;
- Apresentação de sugestões de correção;
- Bloqueio de *commit* em caso de alto risco;
- Permissão de *commit* com alerta em caso de risco médio.
- Validação da mensagem de commit.

Funcionalidades Excluídas

- Interface gráfica;
- Análise de arquivos multimídia;
- Uso de técnicas avançadas como OCR;
- Integração com serviços externos;
- Configuração avançada pelo usuário;

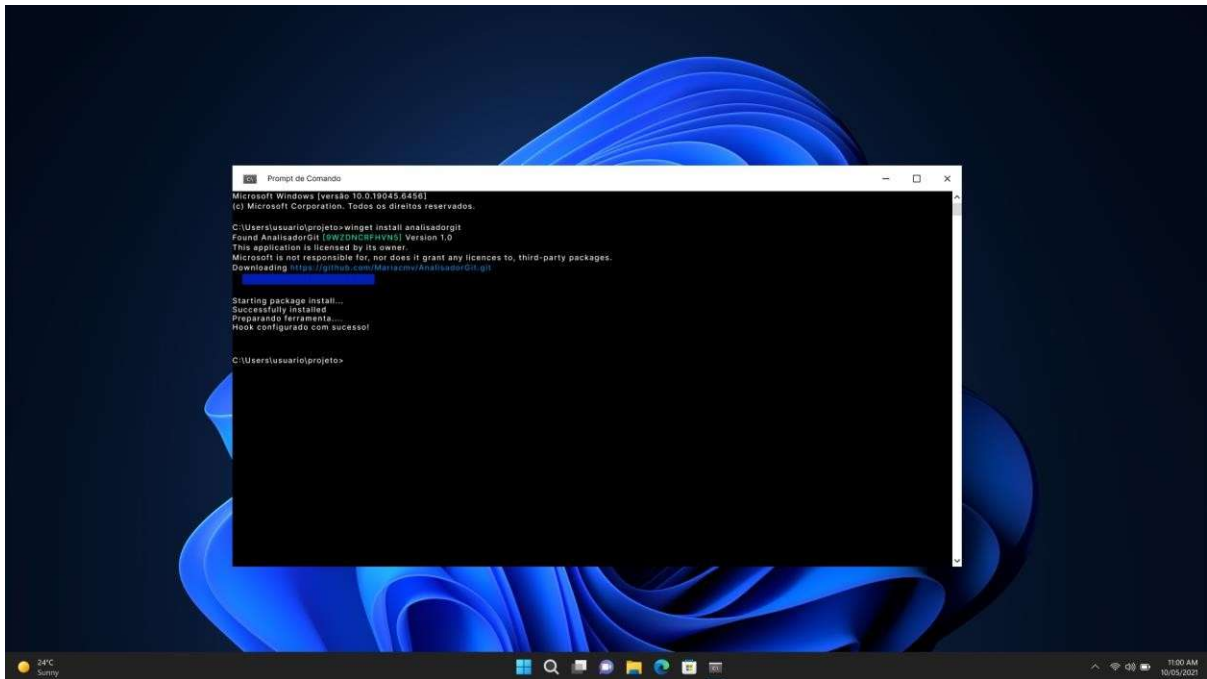
Limitações Conhecidas

- Detecção de falsos positivos;
- Detecção baseada em regras simples;
- Não há diferenciação entre dados reais e fictícios;
- Não cobra todos os tipos de vazamento de dados;
- Não realiza análise histórica de *commits* anteriores;

- Funciona apenas em ambientes de linha de comando;

Protótipo das Telas

Tela 01 - Tela *cmd* instala a ferramenta AnalisadorGit

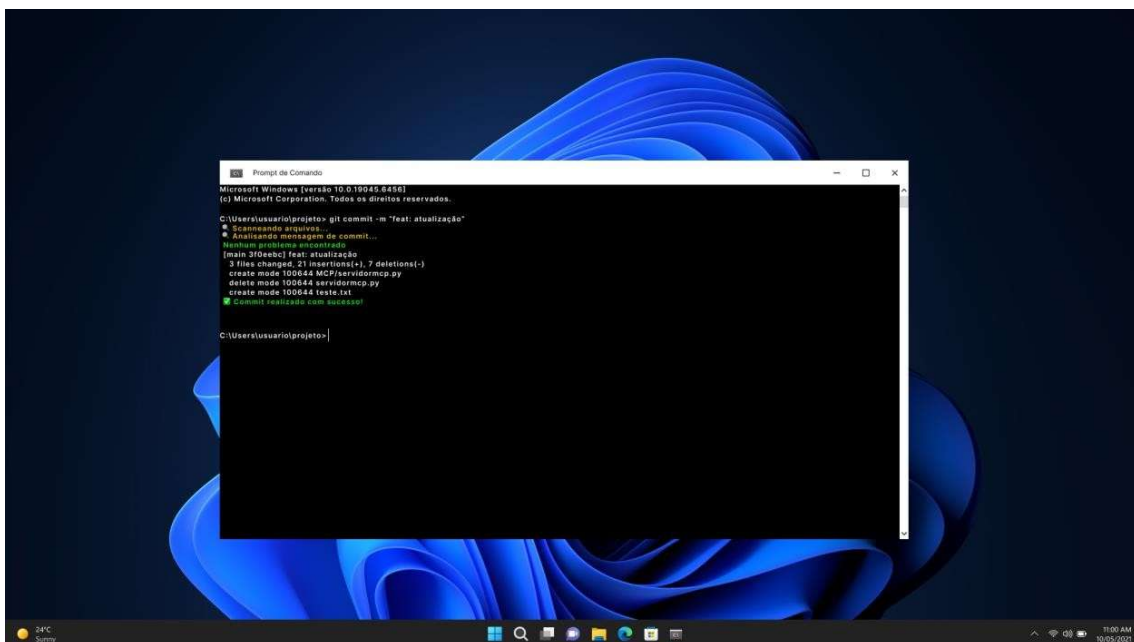


Representa o processo de instalação do sistema AnalisadorGit por meio do terminal.

O usuário executa o comando de instalação e o sistema realiza automaticamente o download e a configuração do hook do Git.

Ao final do processo, é exibida uma mensagem confirmando a instalação e ativação do mecanismo de análise de segurança.

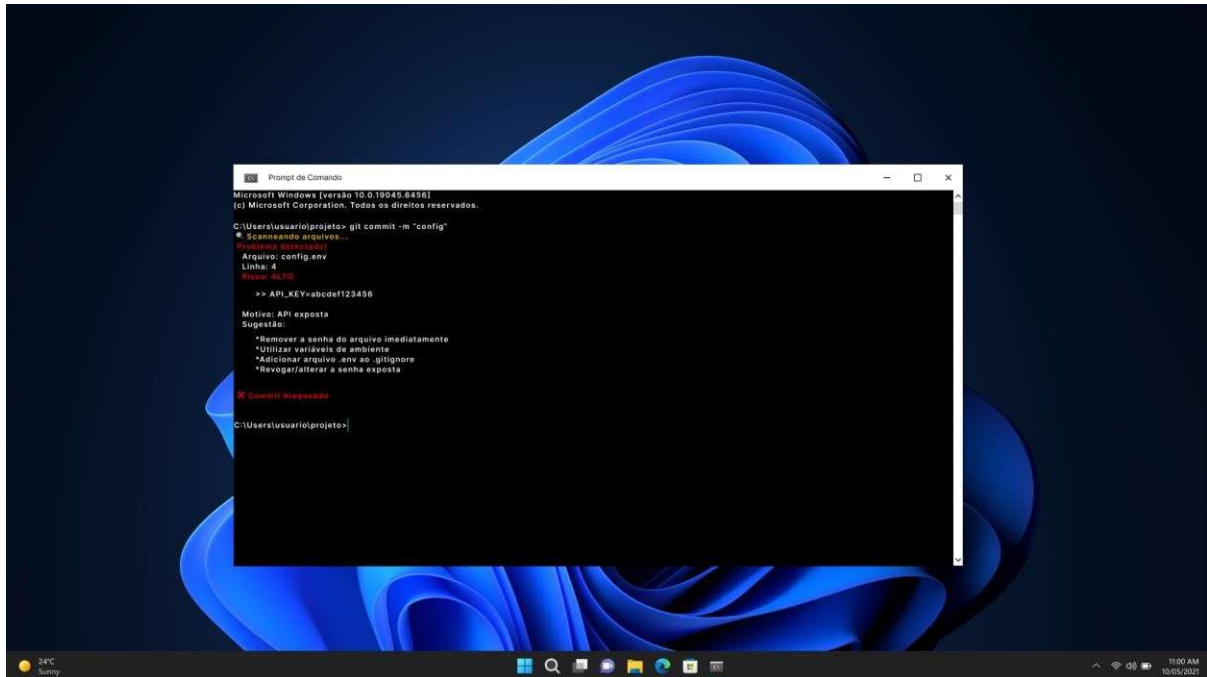
Tela 02 - Tela *cmd commit* com sucesso – sem riscos



Representa a execução de um commit validado com sucesso pelo sistema.

Durante a análise dos arquivos adicionados ao commit, nenhum dado sensível foi identificado. O sistema informa ao usuário que a verificação foi concluída sem riscos, permitindo a continuidade normal do processo de versionamento. Ao final, é exibida uma mensagem confirmando a instalação e a ativação do mecanismo de análise.

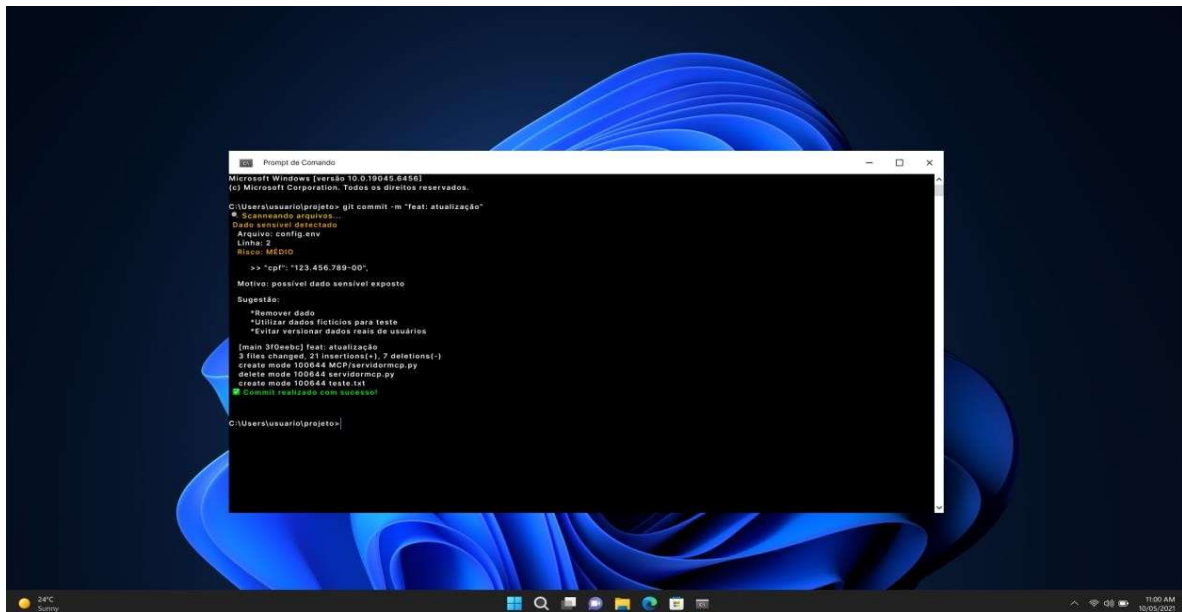
Tela 03 - Tela *cmd commit* defeituoso - alto risco



Representa a detecção de um problema crítico durante a análise de segurança.

O sistema identifica dados sensíveis, como credenciais, tokens ou chaves de API, informando o arquivo e a linha afetada. O risco é classificado como alto e o commit é bloqueado automaticamente para evitar vazamentos de informações.

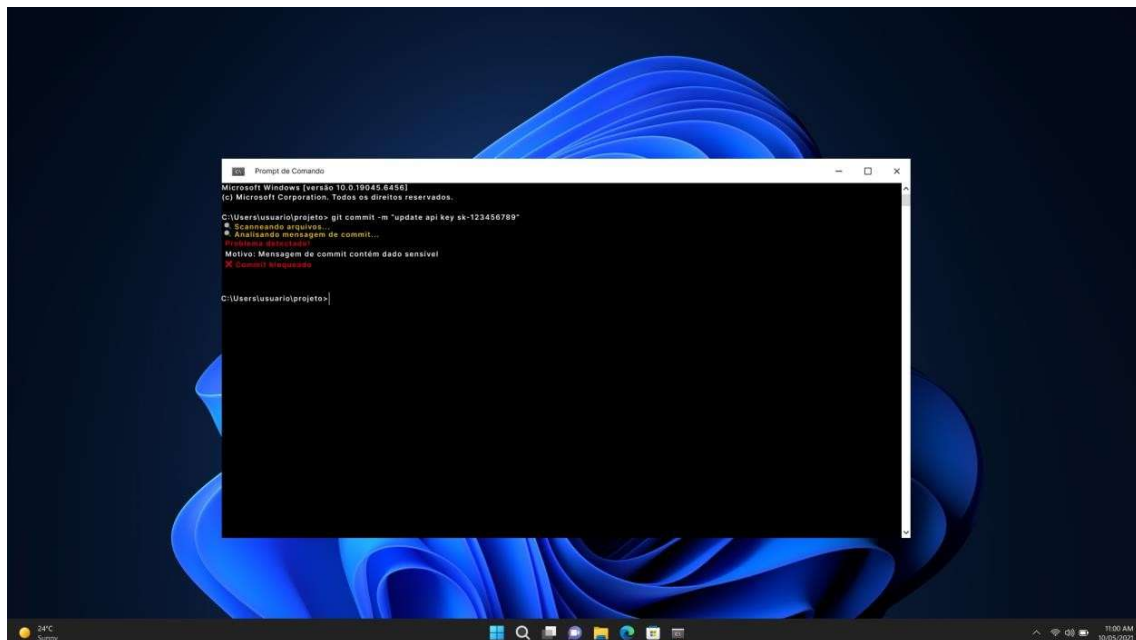
Tela 04 - Tela *cmd commit* com sucesso com informações identificadas - médio risco



Representa a identificação de possíveis dados sensíveis classificados como risco moderado.

O sistema informa o problema encontrado, apresenta sugestões de correção e permite a continuidade do commit mediante alerta ao desenvolvedor.

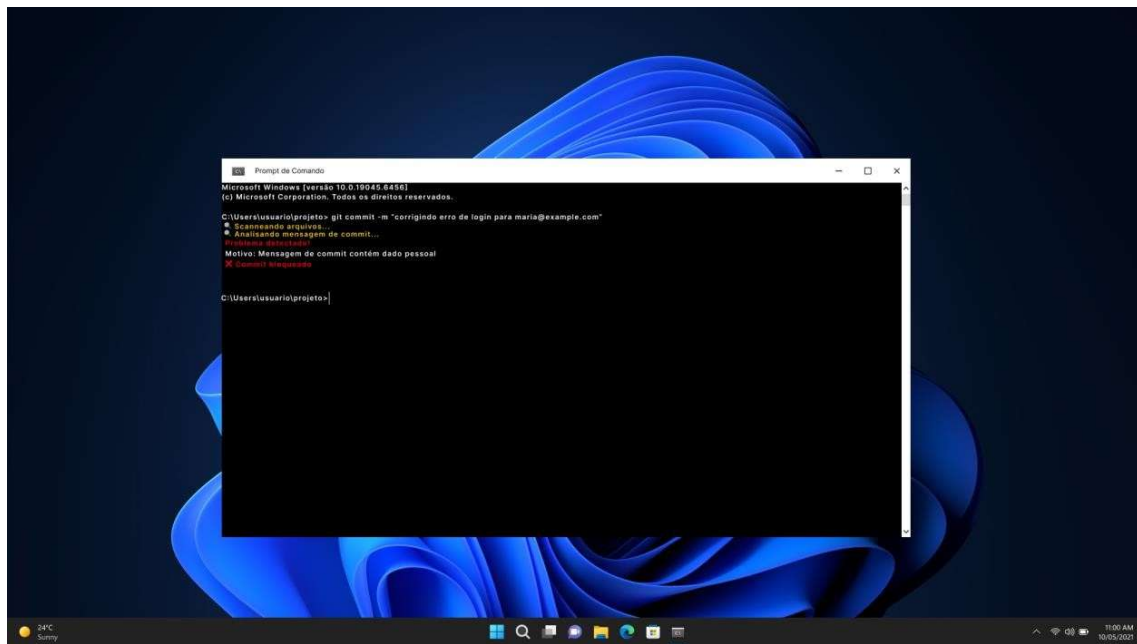
Tela 05 - Tela *cmd commit* com mensagem de *commit* problemática



Representa a validação da mensagem de commit.

O sistema verifica se a mensagem segue os padrões definidos e se não contém informações sensíveis ou inadequadas. Em caso de inconsistência, é exibida uma mensagem de erro e o commit pode ser bloqueado.

Tela 06 - Tela *cmd commit* com mensagem de *commit* problemática



Outro exemplo de inconsistência.

Tipo de Protótipo

O propósito desenvolvido é do tipo de alta fidelidade, pois representa de forma próxima ao funcionamento real do sistema, incluindo fluxos de interação, mensagens exibidas ao usuário e comportamento esperado durante a execução dos commits.

Ferramentas Utilizadas

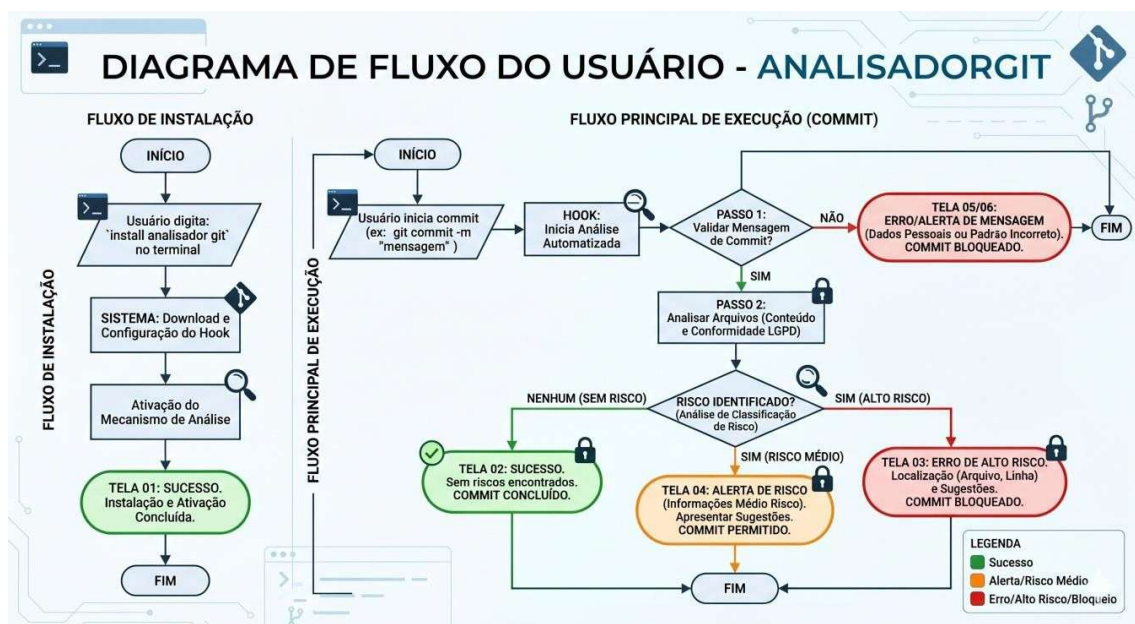
- *Figma* (criação do protótipo);
- *Visual Studio Code* (apoio na estrutura e simulação textual);
- Terminal *CLI* (base para representação da interface do sistema);

Descrição das funcionalidades

O protótipo simula o funcionamento de um sistema de análise de segurança integrado ao fluxo de commits do Git.

Durante a execução de um commit, o sistema realiza automaticamente a análise dos arquivos adicionados, identificando possíveis dados sensíveis. Os problemas encontrados são classificados por nível de risco - alto ou médio, sendo apresentados ao usuário juntamente com informações como arquivo, linha e sugestão de correção. Em casos de alto risco, o commit é bloqueado. Em casos de risco médio, o sistema emite um alerta, permitindo a continuidade do processo. O protótipo também contempla a etapa de instalação do sistema e a validação da mensagem de commit.

Fluxo do Usuário



Fluxo de instalação (Setup)

- Ação do Usuário: Executa o comando de instalação no terminal.
- Processamento: O sistema realiza automaticamente o download e a configuração do *hook* do Git.
- Interface (Tela 01): Exibe a tela de cmd instala a ferramenta AnalisadorGit.
- Resultado: Mensagem confirmando a instalação e a ativação do mecanismo de análise.
- Fim do Fluxo.

Fluxo Principal de Execução (Commit)

- Ação do Usuário: Inicia um commit (ex: `git commit -m "mensagem"`).
- Processamento (Passo 1): O sistema valida a mensagem de commit.
 - Caminho A (Inconsistência na Mensagem): Se a mensagem contém dados pessoais ou foge do padrão.
 - Interface (Telas 05 e 06): Exibe mensagem de erro ou alerta.
 - Resultado: O commit é bloqueado. -> Fim do Fluxo.
 - Caminho B (Mensagem Válida): O sistema prossegue para a análise dos arquivos.
- Processamento (Passo 2): Identificação dos arquivos adicionados e detecção de padrões de dados sensíveis.
- Decisão do Sistema (Classificação de Risco): O sistema analisa os achados e classifica o risco.
 - Caminho C (Sem Riscos Encontrados):
 - Interface (Tela 02): Exibe a tela de commit com sucesso sem riscos.
 - Resultado: Commit realizado com sucesso. -> Fim do Fluxo.
 - Caminho D (Risco Médio Identificado):
 - Interface (Tela 04): Exibe a tela de commit com sucesso com informações identificadas - médio risco.
 - Ação do Sistema: Informa o problema encontrado e apresenta sugestões.
 - Resultado: Emite o alerta para o desenvolvedor, mas permite a continuidade do commit. -> Fim do Fluxo.
 - Caminho E (Alto Risco Identificado):
 - Interface (Tela 03): Exibe a tela de commit defeituoso - alto risco.
 - Ação do Sistema: Informa o arquivo e a linha afetada, e apresenta sugestões de correção.
 - Resultado: O commit é bloqueado para evitar o vazamento de informações sensíveis. -> Fim do Fluxo.

Requisitos do sistema

Requisitos funcionais

- RF01 – Validar mensagens de commit;
- RF02 – Identificar dados sensíveis nos arquivos;
- RF03 – Classificar riscos em níveis;
- RF04 – Exibir sugestões de correção;
- RF05 – Bloquear commits de alto risco;
- RF06 – Permitir commits com risco moderado mediante alerta.

Requisitos não funcionais

- RNF01 – Operar em ambiente CLI;
- RNF02 – Possuir execução rápida durante o commit;
- RNF03 – Ser compatível com Git;
- RNF04 – Exibir mensagens claras e objetivas;
- RNF05 – Possuir fácil instalação.

Casos de Uso

Caso de Uso – Realizar Commit Seguro

Ator Principal: Desenvolvedor

Objetivo: Garantir que commits não contenham dados sensíveis.

Fluxo Principal:

1. O usuário executa um commit;
2. O sistema valida a mensagem;
3. O sistema analisa os arquivos;
4. O sistema identifica possíveis riscos;
5. O commit é aprovado.

Fluxos Alternativos:

- Caso seja identificado risco alto, o commit é bloqueado;
- Caso seja identificado risco moderado, o sistema exibe alerta e permite continuidade.

Guia de estilo

O sistema adota uma interface baseada em linha de comando (CLI), priorizando clareza visual, feedback imediato e facilidade de interpretação das mensagens exibidas ao usuário.

Os padrões visuais definidos incluem:

- Uso de cores para identificação rápida dos níveis de risco:

- Verde (#00FF11): operação segura/sucesso;
 - Amarelo (#FFCC00): atenção;
 - Laranja (#FFA600): risco moderado;
 - Vermelho (#FF0000): risco crítico.
- Uso de símbolos visuais para reforçar o significado das mensagens exibidas no terminal;
 - Estrutura padronizada das mensagens contendo:
 - Tipo do risco;
 - Arquivo afetado;
 - Linha identificada;
 - Sugestão de correção.
 - Linguagem objetiva e direta para reduzir ambiguidades e facilitar a compreensão;
 - Padronização tipográfica utilizando a fonte Inter SemiBold, tamanho 12;
 - Aplicação de princípios de usabilidade:
 - Feedback imediato;
 - Consistência visual;
 - Minimalismo;
 - Clareza das informações;
 - Redução da carga cognitiva.

Melhorias Futuras

- Integração com GitHub e GitLab;
- Implementação de inteligência artificial para redução de falsos positivos;
- Desenvolvimento de dashboard web;
- Análise histórica de commits;
- Configuração personalizada de regras;
- Suporte a múltiplas linguagens de programação.

Conclusão

A prototipagem permitiu validar os principais fluxos de funcionamento do sistema, além de analisar a clareza das interações realizadas com o usuário durante o processo de commit.

A definição das interfaces contribuiu para melhorar a experiência do usuário por meio de mensagens padronizadas, classificação de riscos e sugestões de correção, tornando o processo mais seguro e intuitivo.

O projeto demonstra potencial para auxiliar na prevenção de vazamentos de informações sensíveis durante o desenvolvimento de software, servindo como base para futuras implementações e expansões da solução.

Anexos

Link para o protótipo no figma: [Protótipo AnalisadorGit – Figma](#)